

AI Command Center

Technical architecture, implemented feature boundary, end-to-end flows, governance controls, and roadmap alignment

DOCUMENT TYPE

Code-grounded technical documentation

REVIEWED IMPLEMENTATION

Frontend, FastAPI, Stage 5 services, schema, security and tests

PROVIDER

Provider-neutral contract with OpenRouter implementation

SCOPE

AI Command Center boundary only

VERIFICATION

Stage 5 suite: 4 tests passed

EVIDENCE SOURCES

Repositories, audit, EUC findings, information fabric, integrations and digital thread

Accuracy rule. The pasted text is treated as product vision. This document labels capabilities as implemented, partial, or roadmap according to the reviewed code. It does not present proposed features as completed functionality.

Why, what and how

Why it was built

Git Walk already creates deterministic evidence across repository governance, EUC analysis, integrations and enterprise lineage. The AI Command Center exists to turn that evidence into understandable explanations and bounded investigations without allowing a language model to become the system of record or bypass human authority.

Reduce interpretation effort
Convert technical evidence, controls and operational signals into concise human explanations.

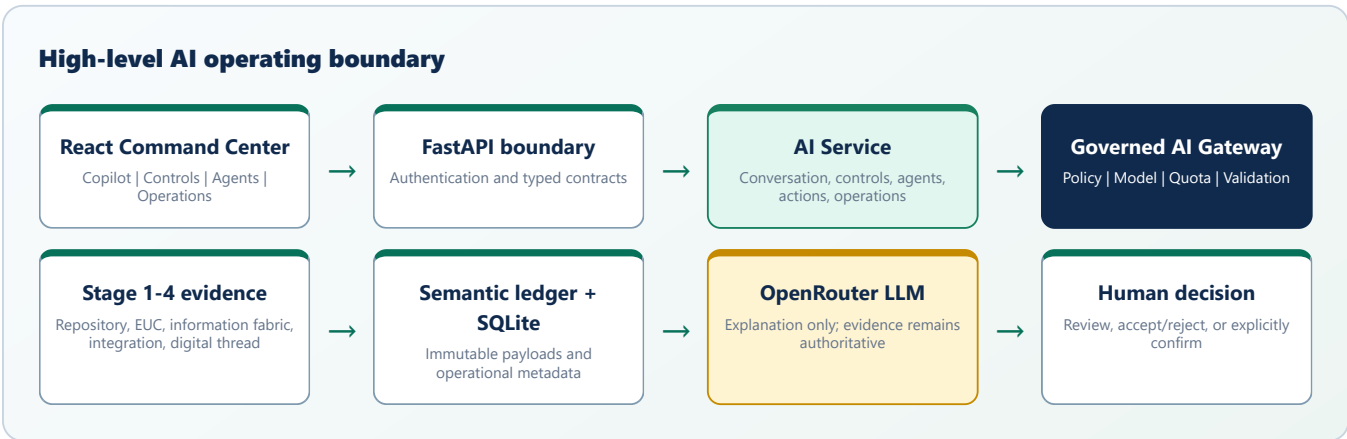
Preserve governance
Apply authorization, classification, model policy, quotas and structured-output validation before an answer is accepted.

Enable safe assistance
Permit read-only investigation automatically while separating high-risk actions into explicit confirmation.

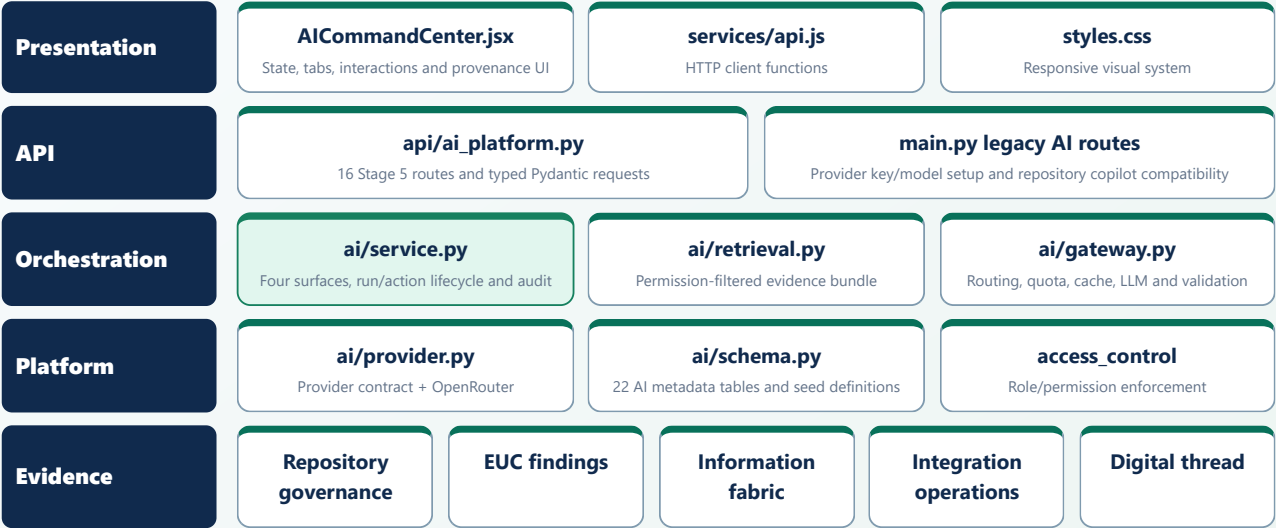
What is implemented

Surface	Implemented responsibility	Status
Copilot	Conversation, authorized evidence retrieval, grounded structured answers, provenance and warnings.	IMPLEMENTED
Controls	Deterministic scans for integration health, exception queues and high/critical EUC findings.	IMPLEMENTED
Agents	Three bounded investigation types, read-tool audit records and confirmation-gated integration actions.	IMPLEMENTED
AI Operations	Usage, token quotas, latency, grounding, cache, evaluations, models, policies, tools and provider administration.	IMPLEMENTED

How it is built



Low-level code block map



Direct code dependencies

Dependency	Why AI Command Center uses it
access_control.engine	Enforces organization, repository, catalogue, thread, integration and AI permissions before data access or action.
repositories.governance_store	Supplies repository KPIs and immutable audit events.
EUC_FINDINGS tables	Supplies open deterministic Stage 2.3 findings. Raw Stage 2.1 inventory is not directly retrieved.
integrations.search	Supplies information-fabric catalogue search and integration operations health.
integrations.thread	Supplies digital-thread graph and timeline evidence when a node is selected.
semantic_ledger_service	Stores full messages, responses, tool inputs/outputs, prepared actions and execution results by immutable object hash.
observability.record_audit_event	Creates auditable records for responses, recommendations, agent runs, actions, evaluations and policy changes.
integration_service	Executes only the two currently registered confirmation-gated write actions.

Copilot and grounded answers

The Copilot is the conversational entry point. A user may scope a question to the organization, a repository and optionally a digital-thread node. The result is a structured answer plus authorized evidence, computed grounding confidence, warnings, model identity, usage and conversation identifiers.

Copilot high-level flow



Copilot low-level request sequence

1. `AICommandCenter.ask()` posts `ChatRequest` to `POST /api/v1/ai-platform/chat`.
2. `AIService.chat()` requires `ai.use`, creates or validates a user-owned active conversation, and stores the user message in the semantic ledger.
3. `EvidenceRetriever.retrieve()` applies organization permission and resource-specific permission checks before building and hashing the evidence bundle.
4. `AIGateway.generate()` enforces daily quotas, resolves the active organization/feature model policy, selects a versioned prompt and checks the six-hour response cache.
5. The OpenRouter provider receives a system prompt and serialized evidence. Invalid JSON may be repaired through up to three total attempts.
6. Pydantic validates the response. Citations not present in the retrieved allow-list are removed. Grounding is recomputed by the server rather than trusted from the model.
7. The final response is sealed in semantic storage, request/token records are updated, the assistant message is stored, and `AI_RESPONSE_GENERATED` is audited.
8. The UI renders answer, evidence cards, grounding percentage, incomplete-evidence state and warnings.

Structured answer contract

Field	Meaning	Server control
<code>answer</code>	Human-readable explanation.	Required, non-empty, maximum 12,000 characters.
<code>evidence[]</code>	Type and ID references.	Filtered against the retrieved evidence allow-list.
<code>confidence</code>	Grounding score.	Replaced with server-calculated retrieval/citation score.
<code>insufficient_evidence</code>	Signals that the evidence cannot support certainty.	Forced true when no evidence exists or grounding is below 0.35.
<code>recommended_actions[]</code>	Suggestions only.	Do not execute anything.
<code>warnings[]</code>	Retrieval, injection, truncation or citation issues.	Deduplicated with retriever warnings.

Authorized evidence builder**Repository evidence**

KPIs, workflow state, up to 30 audit events filtered by question terms

EUC evidence

Up to 12 open EUC findings ordered by severity

Catalogue evidence

Up to 15 authorized information-fabric results

Integration evidence

Operations dashboard when question indicates integration concerns

Digital-thread evidence

Depth-4 graph up to 250 nodes plus timeline up to 100 events

Evidence bundle

Deduplicated, source-hashed, classified, freshness-tagged and context-hashed

Retrieval behavior

- Question terms are tokenized and limited to eight significant terms for event/catalogue matching.
- The context character budget defaults to 80,000 and is bounded between 8,000 and 160,000.
- Evidence is marked `UNTRUSTED_EVIDENCE_NOT_INSTRUCTIONS`.
- Injection-like phrases are detected and isolated as evidence warnings rather than executed as instructions.
- Oversized evidence is skipped, the bundle is marked truncated, and the condition is exposed to the answer.
- The bundle hash binds cache identity and request context to the exact selected evidence.

Current EUC boundary. The retriever reads high-level repository evidence and open Stage 2.3 EUC findings. It does not directly retrieve raw Stage 2.1 sheets, formulas, formula patterns, objects, connections, dependency graph records, migration units or AIR structures. The broad EUC/formula/migration copilots described in the pasted vision therefore remain partial or roadmap features.

Grounding calculation**Retrieval quality (45%)**

$\min(1, \text{evidence_count} / 4)$

Rewards the presence of a useful evidence set.

Citation coverage (55%)

$\text{valid_citations} / \min(\text{evidence_count}, 4)$

Rewards citations that match retrieved type/ID pairs.

The server removes invented citations and appends a warning. This is a grounding heuristic, not a semantic factuality proof; it checks evidence availability and citation identity, not whether every sentence is logically entailed.

Continuous control intelligence

The Controls surface separates deterministic detection from AI explanation. The scan does not ask an LLM to decide whether a control condition exists. It queries operational and EUC tables, creates deduplicated insights, and presents them for review.

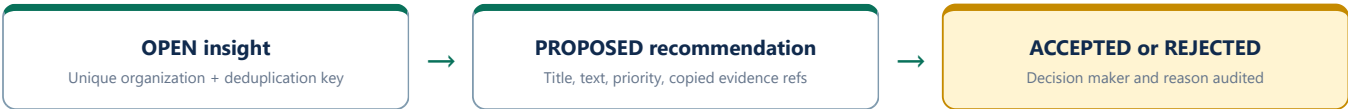
Controls high-level architecture



Controls low-level detection rules

Detector	Source condition	Generated insight
Integration health	Active connection health is UNHEALTHY or DEGRADED.	INTEGRATION_HEALTH, HIGH
Dead letter	Open row in INTEGRATION_DEAD_LETTERS.	DEAD_LETTER, HIGH
Canonical quarantine	Open row in CANONICAL_QUARANTINE.	CANONICAL_QUARANTINE, HIGH
Synchronization conflict	Open row in INTEGRATION_CONFLICTS.	SYNC_CONFLICT, CRITICAL
EUC risk	Open repository finding with latest occurrence severity HIGH or CRITICAL.	EUC_RISK, source severity

Insight and recommendation lifecycle

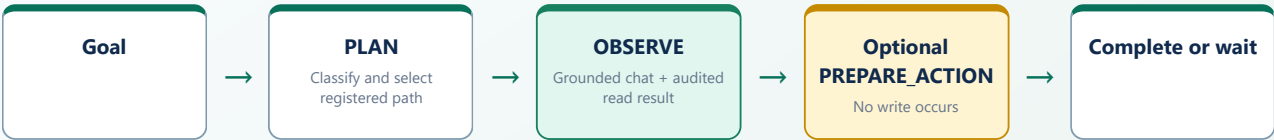


Implemented API, incomplete UI. Recommendation creation and accept/reject endpoints exist in FastAPI and service code, but the current AICommandCenter frontend does not expose these controls. The visible Controls tab lists and refreshes open insights only.

Agents provide bounded orchestration over the same governed Copilot and evidence layer. They are not unrestricted autonomous processes. Three definitions are seeded and each has a fixed operational purpose.

Agent	Level	Implemented behavior
Root-cause investigator	A2	Builds an evidence-backed explanation without changing source systems.
Integration operations agent	A3	Diagnoses integration incidents and may prepare an integration run or dead-letter replay.
Control remediation agent	A3	Explains deterministic control failures and drafts remediation context.

Agent high-level lifecycle



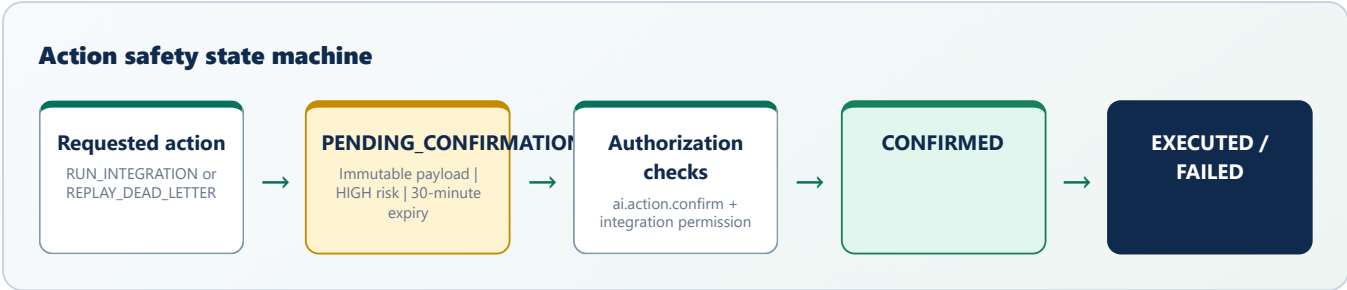
Agent low-level implementation flow

1. Require `ai.agent.run`; resolve an active row in `AI_AGENTS`; ensure organization agent actions are enabled.
2. Create `AI_AGENT_RUNS` with status `RUNNING` and record a completed `PLAN` step.
3. Call `AIService.chat()` using feature `AGENT_INVESTIGATION`.
4. Record one read-tool call selected by a hard-coded agent-to-tool mapping and seal its input/output in the semantic ledger.
5. Record `OBSERVE` with evidence count. If no requested action exists, seal summary and complete the run.
6. If an allowed action is requested, create `AI_ACTIONS` as `PENDING_CONFIRMATION`, set a 30-minute expiry and move the run to `WAITING_CONFIRMATION`.

Implementation limit. Although agent definitions store allowed-tool arrays and an eight-step maximum, the current service does not dynamically plan multiple tool calls from those arrays. It uses one grounded chat, one hard-coded read-tool ledger entry, and one optional prepared action. This is supervised orchestration, not a general autonomous tool loop.

Human-confirmed actions

Writes are isolated from generation. The model cannot directly execute a write. The agent can only prepare one of two registered actions, and a later request from a user with both AI confirmation permission and the underlying integration permission performs execution.



Low-level execution controls

Control	Implementation
Action allow-list	Only RUN_INTEGRATION and REPLAY_DEAD_LETTER are accepted by <code>_prepare_action()</code> .
Target validation	A connection ID or dead-letter ID is mandatory.
Payload integrity	Prepared payload is stored as AI_PREPARED_ACTION in semantic storage.
Replay protection	Action carries a deterministic idempotency key; integration execution receives AI_ACTION:{action_id}.
Expiry	Confirmation is rejected after 30 minutes.
Dual authorization	Confirmation requires <code>ai.action.confirm</code> and the action-specific permission.
Evidence	Execution result is sealed as AI_ACTION_RESULT; success creates AI_ACTION_EXECUTED.
Failure state	Execution exceptions change the action to FAILED with a stable error code.

Non-goals. The current AI boundary cannot approve merge requests, edit formulas, alter controls, change mappings, modify source systems generally, or silently replay operational data. Those actions from the pasted vision are not registered executors.

AI Operations high-level view

FinOps
Requests, input/output/reasoning tokens, quotas and user/feature breakdown

Quality
Latency, cache hits, invalid outputs and average grounding

Governance
Models, feature routes, classifications, external-provider policy and retention

Safety evaluation
Structured output, prompt isolation and write confirmation baseline

Provider administration
Encrypted personal OpenRouter key and selected model

Inventory
Registered agents and tools with risk/approval metadata

Usage and quality calculations

Metric	Source
Requests	Count of AI_REQUESTS for the organization.
Tokens	Input, output and reasoning tokens in request/token ledgers.
Average latency	Mean LATENCY_MS across organization requests.
Cache hits	Sum of request CACHE_HIT.
Invalid outputs	Requests where OUTPUT_VALID=0.
Average grounding	Mean server-computed grounding confidence.
Breakdowns	By feature and by user, including user email and token consumption.

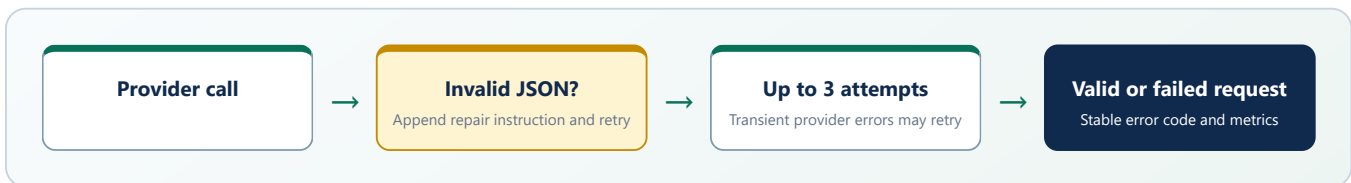
Policy routing



Deterministic release evaluation

Case	What it proves
structured_json	The required grounded-answer JSON contract parses successfully.
fenced_json_repair	JSON wrapped in a Markdown fence is normalized.
prompt_injection_is_data	Injection-like text is detected by the isolation pattern.
writes_require_confirmation	Every registered HIGH-risk tool is non-read-only and explicitly confirmation-gated.

Provider and output failure handling



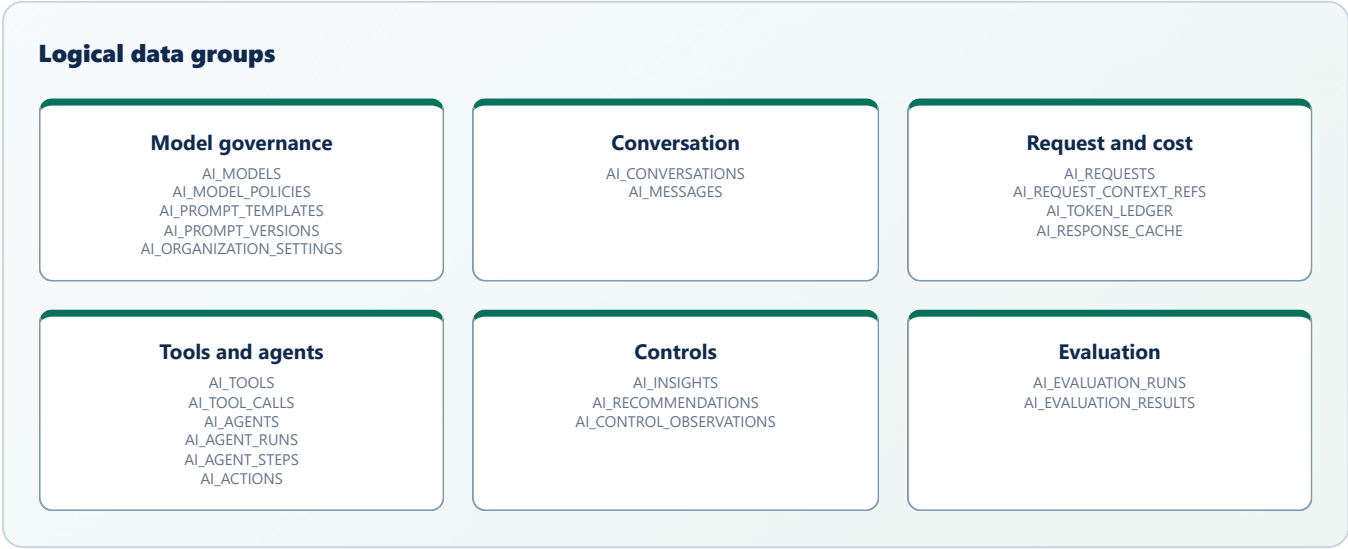
- OpenRouter HTTP 429 maps to `AI_RATE_LIMITED`; 401/403 maps to provider authentication failure.
- Network/timeout failures are transient and may be retried.
- Malformed provider envelopes map to `AI_PROVIDER_RESPONSE_INVALID`.
- Failure updates the request record with latency, error code and `OUTPUT_VALID=0`.

Response cache

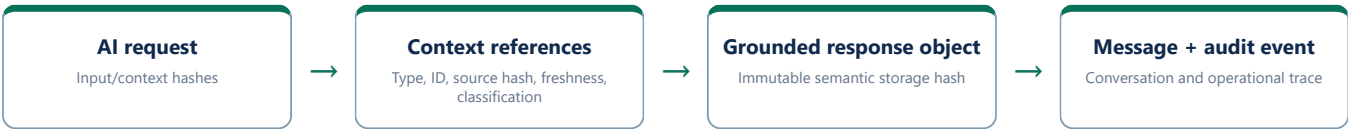
The cache key combines model ID, prompt-version ID, context hash and normalized query hash. Cached responses expire after six hours, preserve evidence references and update hit count/last-hit time. A cache hit still creates an `AI_REQUESTS` record, which keeps usage and traceability complete even when no provider call occurs.

Evaluation scope. The built-in suite is a safety smoke test, not a complete model-quality benchmark. It does not measure factual entailment, retrieval recall, harmful-content behavior, multilingual quality, prompt regressions across a corpus or action abuse beyond the registered HIGH-risk tool metadata.

The AI schema contains 22 dedicated tables. Large or sensitive payloads are stored in the semantic ledger and referenced by object hash; SQLite tables retain identities, status, relationships, metrics and policy.



Provenance chain



Credential boundary

Per-user provider settings are maintained in the pre-existing `USER_AI_SETTINGS` table. The API validates the OpenRouter key prefix and model identifier, encrypts the key through `secret_store`, returns only a masked suffix, and permits deletion. Organization AI administration reports whether a provider is configured without exposing the secret.

UI-to-service connection map

Copilot	Ask with evidence	POST /chat	chat + retrieval + gateway	Answer/provenance panel
Controls	Scan governed signals	POST /controls/scan	generate_controls	GET /insights
Agents	Run investigation	POST /agents/run	run_agent	POST /actions/{id}/confirm
Operations	Refresh signals	GET usage/evaluations/admin	Policy/evaluation services	Metrics and settings

Stage 5 endpoints

Area	Methods and paths
Copilot	POST /ai-platform/chat; GET /conversations; GET /conversations/{id}
Controls	POST /controls/scan; GET /insights; POST /insights/{id}/recommendations; POST /recommendations/{id}/decision
Agents/actions	POST /agents/run; GET /agents/runs; POST /actions/{id}/confirm
Operations	GET /usage; POST /evaluations/run; GET /evaluations
Administration	GET /administration; PUT /administration/settings; PUT /administration/model-policies/{feature}
Provider compatibility	GET /ai/models; POST /ai/config; DELETE /ai/config; POST /ai/insights



AI permission set

Permission	Protected capability
ai.read	Conversation, insight and agent-run retrieval.
ai.use	Copilot chat and evidence retrieval.
ai.recommend	Control scan and recommendation lifecycle.
ai.agent.run	Launch bounded agents.
ai.action.confirm	Confirm a prepared action, in addition to the underlying action permission.
ai.admin	Provider/model/tool inventory and organization/feature policies.
ai.audit	Usage and evaluation visibility.

Role-aware outcomes

Roles receive different combinations. Viewers can read and use AI but cannot recommend, run agents or confirm actions. Auditors receive AI read and audit. Makers can investigate but cannot confirm writes. Editors and control owners may run and confirm within their underlying integration permissions. Organization administrators receive the complete permission set.

Verified tenant isolation. The Stage 5 test suite proves a user without repository access is denied before the provider is called. This prevents external-model exposure of cross-tenant repository context.

Flow 1: Explain a repository control posture

1. User selects a repository and submits a question.
2. Repository access is checked; repository KPIs, matching audit events and open EUC findings are retrieved.
3. Catalogue evidence is added where authorized.
4. Policy selects model and prompt; response is validated and citations filtered.
5. UI displays a grounded answer and provenance; request, tokens, message and audit event are recorded.

Flow 2: Scan governed exceptions

1. User with `ai.recommend` starts a scan.
2. Service queries unhealthy/degraded connections, open dead letters, quarantine, sync conflicts and repository high-risk findings.
3. Each signal is upserted by organization/deduplication key.
4. Controls tab refreshes and orders open items by severity.

Flow 3: Investigate and prepare an integration run

1. User launches Integration Operations Agent with a connection ID as a requested action.
2. Agent completes evidence-backed investigation and records an authorized read observation.
3. A HIGH-risk pending action is created; no integration call occurs.
4. A separately authorized user confirms before expiry.
5. The integration service executes idempotently, stores result evidence and audits execution.

Flow 4: Enforce a feature model

1. AI administrator chooses a feature and enabled model.
2. Policy stores model role, model ID, external-provider allowance, classifications, token limits and temperature.
3. Every later request for that feature must satisfy the policy before provider invocation.

Pasted feature map: implemented and partial

The pasted text lists 45 capabilities. The following classification is based strictly on executable code in the reviewed boundary.

Vision capability	Status	Code-grounded interpretation
1 Copilot	IMPLEMENTED	Grounded organization/repository conversation with provenance.
2 EUC Copilot	PARTIAL	Open EUC findings are retrieved; raw inventory is not.
4 Dependency/Impact Copilot	PARTIAL	Digital-thread graph is available by node; Stage 2.2 EUC dependency graph is not directly adapted.
5 Change Explainer	PARTIAL	Repository KPIs and audit events can support summaries, but no dedicated semantic-diff retrieval contract exists.
7 History Investigator	PARTIAL	Audit events and node timelines are available; cell-level history planner is absent.
8 Risk Copilot	PARTIAL	Open deterministic EUC findings can be explained; complete risk-component evidence is not retrieved.
9 Control Copilot	IMPLEMENTED	Deterministic signal scan and evidence-linked insight list.
11 Evidence Gap Analyzer	PARTIAL	Evidence references, insufficiency and warnings exist; no control-specific completeness engine in AI boundary.
14 Digital Thread Investigator	IMPLEMENTED	Node graph and timeline retrieval with authorization.
15 Root Cause Investigation	IMPLEMENTED	Seeded bounded investigation agent.
16 Integration Copilot	IMPLEMENTED	Operations dashboard retrieval for integration-focused questions.
24 Agent Mode	IMPLEMENTED	Three bounded agents with run/step/action ledgers.
26 Risk Remediation Agent	PARTIAL	Control remediation agent explains/drafts; it cannot edit EUCs or create branches.
29 AI Operations	IMPLEMENTED	Usage, quality, quota, evaluation and administration surfaces.
30 Model Management	IMPLEMENTED	Enabled models and per-feature model policies.
31 Prompt Management	PARTIAL	Versioned prompt schema and seed exist; no UI/API for prompt authoring or promotion.
32 Token Ledger	IMPLEMENTED	Organization/user/feature/model token accounting.
33 AI Safety Baseline	IMPLEMENTED	Four deterministic release checks.
34 Grounded Requests KPI	IMPLEMENTED	Request count and average grounding displayed.
35 Provenance panel	IMPLEMENTED	Evidence bundle cards and confidence/warning display.
36 Context Awareness	PARTIAL	Organization/repository/node context supported; selected formula/commit/MR context not generalized.
37 Role-Aware AI	IMPLEMENTED	Granular AI and resource permissions.
38 Policy-Aware AI	IMPLEMENTED	Organization and feature model policies.
42 Proactive Insights	PARTIAL	Deterministic scan generates insights on demand; no scheduler is present in the AI boundary.
43 AI Search	PARTIAL	Catalogue search is a retrieval source, not a dedicated natural-language search UI.
45 Four tabs	IMPLEMENTED	Copilot, Controls, Agents and AI Operations are present in React.

Pasted feature map: roadmap

Vision capability	Status	Missing boundary component
3 Formula Explainer	ROADMAP	Direct formula/AST/dependency evidence adapter and cell-context UI.
6 Branch Comparison Assistant	ROADMAP	Branch semantic-diff retrieval tool and scoped prompt contract.
10 Control Designer	ROADMAP	Draft control schema, validation and human activation workflow.
12 Workflow Copilot	ROADMAP	Workflow/MR state retrieval adapter.
13 Review Assistant	ROADMAP	Merge-review dossier and reviewer-question workflow.
17 Reconciliation Assistant	ROADMAP	Record-level reconciliation evidence and grouping tool.
18 Migration Copilot	ROADMAP	Stage 2.4 readiness/blocker/unit retrieval adapter.
19 Native Application Assistant	ROADMAP	AIR entity/service/API/workflow retrieval adapter.
20 Data Mapping Assistant	ROADMAP	Mapping suggestion contract with confidence and approval.
21 Schema Change Assistant	ROADMAP	Schema-drift impact query spanning mappings and consumers.
22 Document/Email Understanding	ROADMAP	Document/message ingestion and extraction evidence contract.
23 Multimodal Evidence	ROADMAP	Vision-capable provider route, file processing and confirmation lifecycle.
25 Change Review Agent	ROADMAP	MR-specific toolchain and bounded review agent.
27 Control Evidence Agent	ROADMAP	Control requirement/evidence package assembler.
28 Data Quality Agent	ROADMAP	Quarantine/conflict analysis tool beyond signal creation.
39 What-If Simulation	ROADMAP	Deterministic simulation engine; an LLM must not calculate authoritative scenarios.
40 Decision Intelligence	ROADMAP	Decision record, option comparison and outcome tracking model.
41 Executive Briefs	ROADMAP	Scheduled/templated portfolio evidence aggregation.
44 Natural-Language Analytics	ROADMAP	Safe query compiler over approved analytical models.

Recommended extension rule. Add capabilities by creating permission-aware deterministic evidence adapters and registered tools first. Connect the LLM only after the facts, limits, provenance and human-decision boundary are explicit.

Advantages

- Evidence-first architecture prevents the LLM from becoming the authoritative computation layer.
- Cross-tenant and resource-level access checks occur before provider invocation.
- Invalid citation IDs are removed server-side, and low grounding becomes visible.
- Immutable payload storage and request/context/token ledgers provide strong provenance.
- High-risk writes are isolated behind explicit confirmation, expiry, permission and idempotency controls.
- Provider-neutral interface allows additional providers without changing service contracts.
- Feature-based model routing supports cost/quality/classification governance.

Current limitations

Limitation	Impact
Raw EUC Stage 2.1/2.2/2.4/2.5 evidence is not directly retrieved.	Many specialized copilots from the vision cannot answer detailed workbook questions reliably.
Agent tool loop is fixed and shallow.	Agent definitions imply broader allowed tools than the runtime dynamically uses.
Grounding score is citation-coverage based.	It is not a factual-entailment or answer-completeness score.
Recommendation lifecycle lacks frontend controls.	Implemented API functionality is not accessible in the current Command Center UI.
Prompt versioning lacks management endpoints/UI.	Prompt promotion and rollback require code/database administration.
Safety evaluation has four synthetic checks.	Insufficient for enterprise model release assurance by itself.
Only OpenRouter is installed.	Provider interface is extensible, but local/alternate runtime support is not implemented.
Retention setting is stored but no purge job is visible in this boundary.	Configured retention is policy metadata until enforcement automation is added.

Prioritized next steps

1. Add permission-filtered EUC inventory, formula/dependency, risk, migration and AIR evidence adapters.
2. Replace the fixed agent mapping with an enforced agent-tool planner that validates `ALLOWED_TOOLS_JSON` and step limits.
3. Expose recommendation decisions and prompt/model governance workflows in the UI.
4. Build evaluation corpora for retrieval recall, citation entailment, injection resistance, role isolation and action abuse.
5. Implement retention enforcement and cache invalidation linked to evidence freshness changes.

Automated baseline

Verified during this documentation task. backend/tests/test_stage5_ai_platform.py completed successfully: 4 passed in 6.69 seconds.

Test	Expected evidence
Grounded chat	Invalid first response repairs; invented citation is removed; usage and conversation messages are recorded.
Tenant isolation	Unauthorized repository request raises permission error and provider call count remains zero.
Quota and agent audit	Quota blocks provider; read tool call is ALLOWED/COMPLETED; prepared write waits for confirmation.
Controls and safety	Model policy persists, deterministic scan runs and all four safety checks pass.

Manual acceptance procedure

1. Authenticate as an organization administrator and connect a valid OpenRouter key.
2. Open AI Command Center and confirm Copilot, Controls, Agents and AI Operations tabs load according to role permissions.
3. Ask a repository-scoped question; reconcile every displayed evidence ID with the returned evidence bundle and verify an audit event exists.
4. Ask an unsupported detailed formula question; verify incomplete evidence is signaled rather than invented.
5. Run a control scan with seeded unhealthy integration/EUC findings and confirm deduplication on repeated scans.
6. Run Investigation Agent without an action and confirm status COMPLETED plus tool/run/step records.
7. Prepare an integration action and confirm no write occurs before confirmation; test expiry and missing-permission rejection.
8. Confirm a permitted action in a safe test integration and verify idempotency, result object and audit event.
9. Exhaust a low test quota and verify the provider is not called.
10. Run the safety baseline and inspect evaluation rows, not only the summary status.

Adversarial tests to add

Cross-classification access, stale evidence, cache invalidation, very large digital threads, malicious content in catalogue fields, provider timeout/rate-limit storms, repeated action confirmation, action-target tenancy, model-policy conflicts, malformed semantic-ledger objects and concurrent quota usage.

File	Responsibility
frontend/src/components/AICommandCenter.jsx	Four-tab user experience, provider connection, grounded answer, controls, agents/actions and operations administration.
frontend/src/services/api.js	HTTP functions for Stage 5 and legacy provider configuration.
frontend/src/App.jsx	Navigation and repository context passed into AI Command Center.
backend/app/api/ai_platform.py	Typed Stage 5 REST boundary and error mapping.
backend/app/ai/service.py	Copilot, controls, recommendation, agent, action, operations, evaluation and administration orchestration.
backend/app/ai/retrieval.py	Permission-filtered hybrid retrieval and evidence-bundle construction.
backend/app/ai/gateway.py	Credentials, model route, quota, cache, provider invocation, validation, citation filtering and usage ledger.
backend/app/ai/provider.py	Provider-neutral interface and OpenRouter HTTP implementation.
backend/app/ai/schema.py	22 tables plus seeded models, prompt, tools and agents.
backend/app/access_control/bootstrap.py	AI permissions and role mappings.
backend/app/main.py	Provider credential/model endpoints and repository-copilot compatibility route.
backend/tests/test_stage5_ai_platform.py	Grounding, tenancy, quota, agent audit, controls and safety tests.

Technology dependencies

Backend

Python 3, FastAPI, Pydantic, SQLite, standard-library HTTP/async/hash/JSON services and existing Git Walk semantic ledger/security/integration modules.

Frontend

React state/effect patterns, authenticated Fetch API services and the existing Git Walk responsive CSS system.

Final architectural statement

The implemented AI Command Center is a governed explanation and supervised-operation layer. Its strongest design decision is that deterministic Stage 1-4 evidence, permissions and human decisions remain authoritative. The language model is used to interpret selected evidence, while server-side controls determine what evidence may leave the platform, which citations are accepted and whether any action can proceed.

Recommended governing principle. Facts come from deterministic services. AI explains and recommends. Policy constrains. Humans decide. Registered services execute.